

COMPUTACIÓN DE ALTAS PRESTACIONES (CAP)

Tema 3: OpenMP

Dr. Héctor Martínez Pérez
el2mapeh@uco.es

Dpto. Ingeniería Electrónica y de Computadores
Universidad de Córdoba



La Especificación

Estándar para programación en memoria compartida

<http://www.openmp.org>

Especificaciones:

- Fortran: 1.0 (1997), 2.0 (2000)
- C/C++: 1.0 (1998), 2.0 (2002)
- Fortran/C/C++: 2.5 (2005), 3.0 (2008), 4.0 (2013), **4.5 (2015)**

Antecedentes:

- Estándar ANSI X3H5 (1994)
- HPF, CMFortran

Modelo de Programación

La programación en OpenMP se basa principalmente en directivas del compilador

OpenMP “parallel for”

```
void daxpy(int n, double a, double *x, double *y, double *z) {  
    int i;  
    #pragma omp parallel for  
    for (i=0; i<n; i++)  
        z[i] = a*x[i] + y[i];  
}
```

Ventajas:

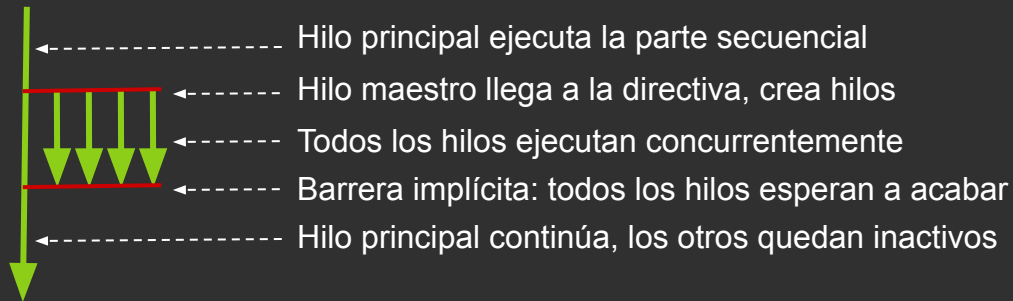
- Facilita la migración (el compilador ignora los #pragma)
- Permite la paralelización incremental
- Permite la optimización por parte del compilador

Modelo de Ejecución

El modelo de ejecución de OpenMP sigue un esquema fork-join

Hay directivas para crear hilos y dividir el trabajo

Esquema



Las directivas definen **regiones paralelas**

Otras directivas: **private**, **shared**, **reduction**, **critical**, **barrier**, ...

Modelo de Ejecución

En OpenMP los hilos inactivos no se destruyen, quedan a la espera de la siguiente región paralela



Los hilos creados por una directiva se llaman **equipo** (team)

Sintaxis

Directivas:

```
#pragma omp <directiva> [clausula [...]]
```

Uso de funciones:

```
#include <omp.h>  
...  
iam = omp_get_thread_num();
```

Compilación:

```
gcc-4.2> gcc -fopenmp myOmp.c
```

```
intel> icc -openmp myOmp.c
```



Ejemplo Simple

`pragma omp parallel for`

```
void daxpy(int n, double a, double *x, double *y, double *z) {  
    int i;  
    #pragma omp parallel for  
    for (i=0; i<n; i++)  
        z[i] = a*x[i] + y[i];  
}
```

- Al llegar a la directiva **parallel** se crean los hilos (si no se han creado antes)
- Las **iteraciones** del bucle se **reparten** entre los hilos
- Por defecto, todas las **variables** son **compartidas**, excepto la variable del bucle (i) que es privada
- Al finalizar se **sincronizan** todos los hilos

Número e Identificador de Hilo

El número de hilos se puede especificar:

- Con la cláusula `num_threads`
- Con la función `omp_set_num_threads()` antes de la región paralela
- Al ejecutar con `OMP_NUM_THREADS`

Funciones útiles:

- `omp_get_num_threads()`: devuelve el número de hilos
- `omp_get_thread_num()`: devuelve el identificador de hilo

Ejemplo



```
omp_set_num_threads(3);
printf("hilos antes = %d\n",omp_get_num_threads());
#pragma omp parallel for
for (i=0; i<n; i++) {
    printf("hilos = %d\n",omp_get_num_threads());
    printf("yo soy %d\n",omp_get_thread_num());
}
```


Directiva: *parallel for*

Se paraleliza el bucle que va a continuación

C/C++

```
#pragma omp parallel for [clausula [...]]  
for (index=first; test_expr; increment_expr) {  
    // cuerpo del bucle  
}
```

OpenMP impone restricciones al tipo de bucle

Bucles Anidados

Hay que poner la directiva antes del bucle a paralelizar

Caso 1

```
#pragma omp parallel for private(j)
for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        // cuerpo del bucle
    }
}
```

Caso 2

```
for (i=0; i<n; i++) {
    #pragma omp parallel for
    for (j=0; j<m; j++) {
        // cuerpo del bucle
    }
}
```

En el primer caso, las iteraciones de (i) se reparten; el mismo hilo ejecuta el bucle (j) completo

En el segundo caso, en cada iteración de (i) se activan y desactivan los hilos; hay (n) sincronizaciones



Tipos de Variables

Se clasifican las variables según su alcance (scope)

- **Privadas**: cada hilo tiene una réplica distinta
- **Compartidas**: todos los hilos pueden leer y escribir

Fuente común de errores: no elegir correctamente el alcance

El alcance se puede modificar con cláusulas añadidas a las directivas:

- **private, shared**
- **reduction**
- **firstprivate, lastprivate**



private, shared

private

```
suma = 0;  
#pragma omp parallel for private(suma)  
for (i=0; i < n; i++) {  
    suma = suma + x[i]*x[i];  
}
```

¿Es correcto?

private, shared

private

```
suma = 0;  
#pragma omp parallel for private(suma)  
for (i=0; i < n; i++) {  
    suma = suma + x[i]*x[i];  
}
```

¿Es correcto? **NO**, tras el bucle sólo existe la suma del hilo principal (valor 0). Además, las copias de cada hilo no se inicializan

private, shared

shared

```
suma = 0;  
#pragma omp parallel for shared(suma)  
for (i=0; i < n; i++) {  
    suma = suma + x[i]*x[i];  
}
```

¿Es correcto?

private, shared

shared

```
suma = 0;  
#pragma omp parallel for shared(suma)  
for (i=0; i < n; i++) {  
    suma = suma + x[i]*x[i];  
}
```

¿Es correcto? **NO**, condición de carrera al leer/escribir la variable **suma**

reduction

Para realizar reducciones con operadores conmutativos y asociativos (+, *, &, ...)

```
reduction(redn_oper: var_list)
```

```
suma = 0;  
#pragma omp parallel for reduction(+:suma)  
for (i=0; i < n; i++) {  
    suma = suma + x[i]*x[i];  
}
```

Cada hilo realiza una porción de la suma, al final se combinan en la suma total

Como una variable privada, pero se inicializa correctamente

firstprivate, lastprivate

Las variables privadas se crean sin un valor inicial y tras el bloque *parallel* quedan indefinidas:

- *firstprivate*: inicializa al valor del hilo principal
- *lastprivate*: se queda con el valor de la “última” iteración

firstprivate, lastprivate

```
alpha = 5.0;
#pragma omp parallel for firstprivate(alpha) lastprivate(i)
for (i=0; i<n; i++) {
    z[i] = alpha*x[i];
}
k = i;
/* i tiene el valor n */
```

El comportamiento por defecto intenta evitar copias innecesarias

Garantizar Suficiente Trabajo

La paralelización de bucles supone un *overhead*: activación y desactivación de hilos, sincronización...

En bucles muy sencillos, el overhead puede ser mayor que el tiempo de cálculo

Cláusula *if*

```
#pragma omp parallel for if (n>800)
for (i=0; i<n; i++)
    z[i] = a*x[i] + y[i];
```

Si la expresión es falsa, el bucle se ejecuta secuencialmente

Esta cláusula se podría usar también para evitar dependencias de datos detectadas en tiempo de ejecución

Planificación

El reparto de trabajo a los hilos puede ser:

- **Estático**: las iteraciones se asignan a hilos a priori
- **Dinámica**: la asignación se adapta a la ejecución actual

Sintaxis de la cláusula de planificación:

```
schedule (type[, chunk])
```

- **static** (sin *chunk*): a cada hilo se le asigna estáticamente un rango aproximadamente igual
- **static**: asignación cíclica (*round-robin*) de rangos de tamaño *chunk*
- **dynamic** (*chunk* opcional, por defecto 1): se van asignando según se piden (*first-come, first-served*)
- **guided** (*chunk* mínimo opcional): como *dynamic* pero el tamaño de rango va decreciendo
- **runtime**: se especifica en tiempo de ejecución con la variable de entorno OMP_SCHEDULE



Directiva *parallel*

Se paraleliza el bloque que va a continuación

C/C++

```
#pragma omp parallel [clause [clause ...]]  
{  
    //bloque  
}
```

Las cláusulas permitidas son: *private*, *shared*, *default*, *reduction*, *if*, *copying*

Funcionamiento de la Directiva *parallel*

Se activan los hilos y se replica el código del bloque

Ejecución replicada

```
#pragma omp parallel private(i)
for (i=0; i<10; i++) {
    printf("Iteración %d\n", i);
}
```

Con 2 hilos, se imprimirán 20 líneas

Reparto de iteraciones

```
#pragma omp parallel for
for (i=0; i<10; i++) {
    printf("Iteración %d\n", i);
}
```

Se imprimen 10 líneas, repartiéndose entre los hilos

Variables *threadprivate*

Declara variables globales privadas a nivel de hilo

threadprivate

```
int initialized = 0; /* variable global */
#pragma omp threadprivate(initialized)
...
#pragma omp parallel
{
    procesa_array();
}
...
void procesa_array() {
    if (!initialized) { initialized=1; /* inicializa */ }
    /* procesamiento normal */
}
```

Estas variables se mantienen de una región paralela a otra - los hilos no se destruyen (a no ser que cambie el número de hilos)



Variables *threadprivate*

Ejemplo

```
int a, b, i, tid;
float x;

#pragma omp threadprivate(a, x)
int main () {
    omp_set_dynamic(0); /* Explicitly turn off dynamic threads */

    #pragma omp parallel private(b,tid)
    {
        tid = omp_get_thread_num();
        a = tid;
        b = tid;
        x = 1.1 * tid + 1.0;
        printf("Thread %d:  a,b,x= %d %d %f\n",tid,a,b,x);
    } /* end of parallel section */

    #pragma omp parallel private(tid)
    {
        tid = omp_get_thread_num();
        printf("Thread %d:  a,b,x= %d %d %f\n",tid,a,b,x);
    } /* end of parallel section */
}
```



Variables *threadprivate*

Ejemplo

```
int a, b, i, tid;
float x;

//#pragma omp threadprivate(a, x) ¿Qué ocurre?
int main () {
    omp_set_dynamic(0); /* Explicitly turn off dynamic threads */

    #pragma omp parallel private(b,tid)
    {
        tid = omp_get_thread_num();
        a = tid;
        b = tid;
        x = 1.1 * tid + 1.0;
        printf("Thread %d: a,b,x= %d %d %f\n",tid,a,b,x);
    } /* end of parallel section */

    #pragma omp parallel private(tid)
    {
        tid = omp_get_thread_num();
        printf("Thread %d: a,b,x= %d %d %f\n",tid,a,b,x);
    } /* end of parallel section */
}
```



Cláusula *copyin*

En el ejemplo anterior, las copias privadas de la variable initialized se inicializan según la declaración (o el constructor en C++)

La cláusula *copyin* permite pasar el valor que tiene el hilo principal a todas las copias privadas

copying

```
int n;  
/* variable global */  
#pragma omp threadprivate(n)  
...  
fread(&n, ...);  
/* el hilo principal lee de fichero */  
...  
#pragma omp parallel copyin(n)  
{  
  procesa_array(n,...);  
}
```

Reparto de trabajo (*for/do*)

Reparte de forma automática las iteraciones del bucle

bucle compartido

```
#pragma omp parallel
{
    ...
    #pragma omp for
    for (i=1; i<n; i++)
        b[i] = (a[i] + a[i-1]) / 2.0;
}
```

El bucle se comparte entre los hilos en vez de hacerlo replicado

Las directivas parallel y for se pueden combinar en una `pragma omp parallel for`

Cláusula `nowait`

Cuando hay varios bucles independientes dentro de una región paralela, `nowait` evita la barrera implícita

Bucles sin barrera

```
void a8(int n, int m, float *a, float *b, float *y, float *z)
{
    int i;
    #pragma omp parallel
    {
        #pragma omp for nowait
        for (i=1; i<n; i++)
            b[i] = (a[i] + a[i-1]) / 2.0;
        #pragma omp for
        for (i=0; i<m; i++)
            y[i] = sqrt(z[i]);
    }
}
```

Construcción sections

Para trozos de código independientes difíciles de paralelizar

- Individualmente suponen muy poco trabajo, o bien
- Cada fragmento es inherentemente secuencial

Puede también combinarse con `parallel`

sections

```
#pragma omp parallel sections
{
  #pragma omp section
  Xaxis();
  #pragma omp section
  Yaxis();
  #pragma omp section
  Zaxis();
}
```

Un hilo puede ejecutar más de una sección

Construcción sections

Ejemplo

```
int main () {
    int thid;
    int a;
    for (int i = 0; i < 10; i++) {
        a= 0;
        printf("litr %d ----->\n", i);
        #pragma omp parallel sections
        {
            #pragma omp section
            {
                a += 1;
                thid = omp_get_thread_num();
                printf("Section 1 - Thread id=%d, a=%d\n", thid, a);
            }
            #pragma omp section
            {
                a += 1;
                thid = omp_get_thread_num();
                printf("Section 2 - Thread id=%d a=%d\n", thid, a);
            }
        }
    }
}
```



Construcción sections

Ejemplo

```
int main () {
    int thid;
    int a;
    for (int i = 0; i < 10; i++) {
        a= 0;
        printf("litr %d ----->\n", i);
        #pragma omp parallel sections private(a)
        {
            #pragma omp section
            {
                a += 1;
                thid = omp_get_thread_num();
                printf("Section 1 - Thread id=%d, a=%d\n", thid, a);
            }
            #pragma omp section
            {
                a += 1;
                thid = omp_get_thread_num();
                printf("Section 2 - Thread id=%d a=%d\n", thid, a);
            }
        }
    }
}
```



Construcción sections

Ejemplo

```
int main () {
    int thid;
    int a;
    for (int i = 0; i < 10; i++) {
        a = 0;
        printf("ltr %d ----->\n", i);
        #pragma omp parallel sections firstprivate(a)
        {
            #pragma omp section
            {
                a += 1;
                thid = omp_get_thread_num();
                printf("Section 1 - Thread id=%d, a=%d\n", thid, a);
            }
            #pragma omp section
            {
                a += 1;
                thid = omp_get_thread_num();
                printf("Section 2 - Thread id=%d a=%d\n", thid, a);
            }
        }
    }
}
```



Construcción single

Fragmentos de código que deben ejecutarse por un solo hilo

Hay una barrera implícita al final del bloque

single

```
#pragma omp parallel
{
    #pragma omp single nowait
    printf("Empieza work1\n");
    work1();
    #pragma omp single
    printf("Finalizando work1\n");
    #pragma omp single nowait
    printf("Terminado work1, empieza work2\n");
    work2();
}
```


Condición de Carrera

El siguiente ejemplo ilustra una condición de carrera

condición de carrera

```
cur_max = -100000;  
#pragma omp parallel for  
for (i=0; i<n; i++) {  
    if (a[i] > cur_max) {  
        cur_max = a[i];  
    }  
}
```

Secuencia con resultado incorrecto:

1. Hilo 0: lee $a[i] = 20$, lee $cur_max=15$
2. Hilo 1: lee $a[i] = 16$, lee $cur_max=15$
3. Hilo 0: comprueba $a[i] > cur_max$, escribe $cur_max=20$
4. Hilo 1: comprueba $a[i] > cur_max$, escribe $cur_max=16$

Directiva `critical`

En el ejemplo anterior, el caso en exclusión mutua a la variable `cur_max` evita la condición de carrera

`critical`

```
cur_max = -100000;
#pragma omp parallel for
for (i=0; i<n; i++) {
    #pragma omp critical
    if (a[i] > cur_max) {
        cur_max = a[i];
    }
}
```

Cuando un hilo llega al bloque `if` (la sección crítica), espera hasta que no hay otro hilo ejecutándolo al mismo tiempo



Directiva `critical`

Al añadir un nombre se permite tener varias secciones críticas sin relación entre ellas

`critical` con nombre

```
cur_max = -100000;
cur_min = 100000;
#pragma omp parallel for
for (i=0; i<n; i++) {
    if (a[i] > cur_max) {
        #pragma omp critical (maximo)
        if (a[i] > cur_max)
            cur_max = a[i];
    }
    if (a[i] < cur_min) {
        #pragma omp critical (minimo)
        if (a[i] < cur_min)
            cur_min = a[i];
    }
}
```



Directiva `atomic`

Operaciones atómicas de lectura y modificación

`atomic`

```
#pragma omp parallel for shared(x, index, n)
for (i=0; i<n; i++) {
    #pragma omp atomic
    x[index[i]] += work1(i);
}
```

El código es mucho más eficiente que con `critical` y permite actualizar elementos de `x` en paralelo

Cerrojos: rutinas *_lock

Funcionamiento muy similar a los mutex de POSIX

```
omp_lock_t lck;  
int id;  
omp_init_lock(&lck);  
#pragma omp parallel shared(lck) private(id)  
{  
    id = omp_get_thread_num();  
    omp_set_lock(&lck);  
    /* solo un hilo a la vez puede ejecutar este printf */  
    printf("Mi id de hilo es %d.\n", id);  
    omp_unset_lock(&lck);  
    while (! omp_test_lock(&lck)) {  
        skip(id); /* aun no tenemos el cerrojo, por lo que tenemos que hacer otra cosa */  
    }  
    work(id); /* ahora tenemos el cerrojo y podemos hacer el trabajo */  
    omp_unset_lock(&lck);  
}  
omp_destroy_lock(&lck);
```



Directiva `barrier`

Al llegar a una barrera, los hilos esperan a que lleguen todos

`barrier`

```
#pragma omp parallel private(index)
{
    index = generate_next_index();
    while (index>0) {
        add_index(index);
        index = generate_next_index();
    }
    #pragma omp barrier
    index = get_next_index();
    while (index>0) {
        process_index(index);
        index = get_next_index();
    }
}
```

Se suele usar para asegurar que una fase la han terminado todos antes de pasar a la siguiente fase



Directiva master

Identifica un bloque de código dentro de una región paralela que se ha de ejecutar solo por el hilo principal (master)

master

```
#pragma omp parallel for
for (i=0; i<n; i++) {
    calc1();
    /* realizar cálculos */
    #pragma omp master
    printf("Resultados intermedios: ...\n", ...);
    calc2();
    /* realizar más cálculos */
}
```

Diferencias con `single`:

- No se requiere que todos los hilos alcancen esta construcción
- No hay barrera implícita (los otros hilos simplemente se saltan ese código)



OMP_NUM_THREADS: Fijar el número de hilos

OMP_PLACES: Asignar cada hilo a un core del procesador

OMP_PROC_BIND: Fijar hilos a cores

OMP_NESTED: Desactivar/Activar paralelismo anidado

GOMP_CPU_AFFINITY: Igual que OMP_PLACES, OJO: GNU no OpenMP

<https://www.openmp.org/spec-html/5.0/openmp.html>

Saxpy: $z = \alpha x + y$

Paralelizar con openMP la implementación de la práctica anterior (Tema 1)

Are You Doing All You Can?

